INFORME DE LABORATORIO 3

Diseño y Validación de Gramáticas

Gramáticas Libres de Contexto

Antecedente	Información
Asignatura	Teoría de la Computación INFO1148
Carrera	Ingeniería Civil en Informática
Docente	Marcos Lévano
Integrantes	Franco Oyarzo Calisto Vicente Rivera Sánchez (jefe de pareja)
Fecha de entrega	25 de octubre de 2026
Herramientas	JFLAP 7.1, Java 21 y Python 3
Repositorio	github.com/Frante23/info1148-lab3

1. Introducción

En este laboratorio diseñamos una gramática libre de contexto para un lenguaje que permite asignar valores, realizar operaciones aritméticas y mostrar variables. Definimos sus terminales y no terminales, construimos las producciones y comprobamos tanto instrucciones aisladas como el programa completo de referencia. Después desarrollamos derivaciones por la izquierda, construimos el árbol de `z = x + y #` y analizamos los errores de las cadenas indicadas en la pauta. Finalmente incorporamos declaraciones de variables con `entero` y `real`, manteniendo las instrucciones anteriores y verificando los cambios con la misma gramática utilizada en el informe.

Método y notación

Separamos la identificación de los lexemas del análisis de su orden. Por ejemplo, `temporal` es el lexema de un token `ID`; `1.345` es el lexema de un token `NUM`. La gramática trabaja con esas categorías finitas. Las anotaciones ID[z] y NUM[20] conservan el texto de la fuente para leer las derivaciones; el atributo entre corchetes no añade un terminal nuevo ni una producción.

En JFLAP representamos cada terminal mediante un carácter y cada no terminal mediante una letra mayúscula. La tabla del anexo A permite pasar de las producciones del informe a los archivos `.jff`. Por ejemplo, `i=i+i#` representa los tokens de `z = x + y #`; se conservan, por separado, sus lexemas. Las pruebas exigen consumir la cadena completa.

2. Desarrollo

2.1. Pregunta 1 Componentes de la gramática

a) Símbolos terminales

Categoría	Terminales	Tratamiento
Palabras reservadas	`Begin`, `Programa`, `End`, `salida`	Se reconocen con esa combinación exacta de mayúsculas y minúsculas.
Operadores	`=`, `+`, `-`, `*`, `/`, `%`	El primero asigna; los otros cinco se ubican entre dos operandos.
Símbolos especiales	`#`	Termina cada instrucción, pero no el encabezado ni `End`.
Identificadores	`ID`	Categoría para `x`, `z2`, `temporal` y `operativo_01_02`, entre otros.
Números	`NUM`	Categoría para enteros y decimales sin signo, incluidos `0`, `20` y `1.345`.

Los espacios y saltos de línea separan lexemas, pero no son terminales de la GLC. Los comentarios `/* ... */` se descartan en la etapa léxica. El guion bajo y el punto decimal pertenecen a la escritura de `ID` y `NUM`; no se reciben como tokens separados en el análisis sintáctico.

b) Símbolos no terminales

Utilizamos once no terminales. Los nombres descriptivos y sus abreviaturas son:

S	`<programa>`	L	`<lista_instrucciones>`
I	`<instruccion>`	A	`<asignacion>`
C	`<asignacion_simple>`	R	`<asignacion_aritmetica>`
V	`<operando>`	F	`<operador>`
O	`<salida>`	D	`<identificador>`
N	`<numero>`		

Los nombres entre ángulos y las letras representan los mismos símbolos. Su función se explica en la pregunta 2.

c) Símbolo inicial

El símbolo inicial es $S =$ `<programa>`, porque el lenguaje principal contiene programas completos. Para estudiar las instrucciones de las preguntas 3 y 4 usamos derivaciones desde $I =$ `<instruccion>`. Distinguimos:

$$L_S = \{w \in \Sigma^* \mid S \Rightarrow^* w\}, \qquad L_I = \{w \in \Sigma^* \mid I \Rightarrow^* w\}.$$

Una instrucción válida no es, por sí sola, un programa completo: los tokens de `x = 10 #` pertenecen a L_I, pero necesitan encabezado y cierre para pertenecer a L_S.

d) Definición formal de la gramática

La gramática base es $G = (V, \Sigma, P, S)$, donde:

$$V = \{\texttt{<programa>}, \texttt{<lista_instrucciones>}, \texttt{<instruccion>},\\ \texttt{<asignacion>}, \texttt{<asignacion_simple>}, \texttt{<asignacion_aritmetica>},\\ \texttt{<operando>}, \texttt{<operador>}, \texttt{<salida>}, \texttt{<identificador>}, \texttt{<numero>}\},$$

$$\Sigma = \{\texttt{Begin}, \texttt{Programa}, \texttt{End}, \texttt{salida}, \texttt{ID}, \texttt{NUM},\\ \texttt{=}, \texttt{\#}, \texttt{+}, \texttt{-}, \texttt{*}, \texttt{/}, \texttt{\%}\}.$$

En las producciones usamos las abreviaturas de la tabla anterior: en particular, la letra `V` representa `<operando>`, mientras que V en la cuádrupla nombra el conjunto completo de no terminales. El conjunto P contiene las 19 alternativas de la pregunta 2 y el símbolo inicial es $S =$ `<programa>`. Se cumple $V \cap \Sigma = \varnothing$, con once no terminales y trece tipos de terminal.

Cada producción tiene un solo no terminal en su lado izquierdo. Por ejemplo, $R \to D = VFV\#$ se aplica independientemente de los símbolos vecinos; esa es la condición que hace libre de contexto a esta gramática.

e) Análisis de identificadores y números

No enumeramos `x`, `y`, `temporal` ni cada nombre futuro como terminales distintos. Una lista de ese tipo solo describiría los ejemplos conocidos. Tampoco enumeramos los números: siempre se podría escribir un entero de más dígitos o un decimal diferente. Usamos dos categorías con reglas léxicas explícitas:

Token	Patrón	Ejemplos
`ID`	`[A-Za-z][A-Za-z0-9_]*`	`x`, `z2`, `Variable_A2`.
`NUM`	`[0-9]+(\.[0-9]+)?`	`0`, `20`, `1.345`.

En el patrón numérico, el punto escapado es literal; el grupo decimal es opcional. Si está presente, exige al menos un dígito a cada lado. Así, `.5` y `1.` se rechazan. No incluimos signo unario ni exponentes. Aunque el enunciado los llama positivos, aceptamos `0` porque aparece expresamente en el programa. También admitimos ceros iniciales: no hay una restricción que los prohíba.

La etapa léxica toma el lexema completo antes de clasificarlo. Por eso `salida` es reservada y no puede usarse como nombre de variable, mientras que `salida2` sí es `ID`. Adoptamos letras ASCII para precisar el alfabeto implementado; los ejemplos exigidos están dentro de ese alcance. El reconocimiento distingue mayúsculas: `Begin` es reservada y `begin` no lo es.

Los comentarios se reconocen desde `/*` hasta el primer `*/`, sin anidamiento. Se eliminan sin unir palabras: `x/* c */y` deja dos identificadores. Un comentario sin cierre produce un error léxico antes de consultar las producciones. Formalmente, el código fuente se acepta cuando su tokenización está definida y la secuencia resultante pertenece a L_S o L_I, según el contexto indicado.

2.2. Pregunta 2 Construcción de la Gramática Libre de Contexto

a) Producciones propuestas

Cada barra separa alternativas; no representa un terminal del lenguaje.

Regla	Producción	Alternativas
P1	$S \rightarrow$ `Begin` `Programa` D L `End`	1
P2	$L \rightarrow I\ L \mid I$	2
P3	$I \rightarrow A \mid O$	2
P4	$A \rightarrow C \mid R$	2
P5	$C \rightarrow D$ `=` V `#`	1
P6	$R \rightarrow D$ `=` $V\ F\ V$ `#`	1
P7	$V \rightarrow D \mid N$	2
P8	$F \rightarrow$ `+` | `-` | `*` | `/` | `%`	5
P9	$O \rightarrow$ `salida` D `#`	1
P10	$D \rightarrow$ `ID`	1
P11	$N \rightarrow$ `NUM`	1
	Total de alternativas	**19**

b) Función de cada no terminal

Símbolo	Función dentro de la gramática
S	Exige `Begin`, `Programa`, un nombre, instrucciones y `End`, en ese orden.
L	Genera una o más instrucciones: $L \rightarrow IL$ continúa; $L \rightarrow I$ termina. Sin $L \rightarrow \varepsilon$, el programa vacío se rechaza.
I	Selecciona una asignación o una salida.
A	Distingue explícitamente entre asignación simple y aritmética.
C	Asigna un solo operando, como `x = 20 #` o `temporal = x #`.
R	Asigna el resultado de exactamente una operación binaria, como `z = x + y #`.
V	Permite un identificador o un número como operando.
F	Restringe la operación a uno de los cinco símbolos permitidos.
O	Exige un identificador después de `salida` y luego el terminador.
D	Introduce un token identificador, con su lexema asociado.
N	Introduce un token numérico, con su lexema asociado.

No añadimos cadenas de operaciones ni paréntesis: `x = y + z * t #` queda fuera del alcance definido. Con una sola operación por asignación no es necesario decidir precedencia ni asociatividad. Esta decisión admite todas las operaciones del programa de referencia.

Justificación de la construcción

La lista obliga a tener instrucciones y permite cualquier cantidad finita. El caso base $L \to I$ genera una instrucción. Si una lista puede generar n instrucciones, $L \to IL$ permite anteponer otra y generar $n+1$. Por inducción, se admite toda cantidad finita mayor o igual que uno. En sentido inverso, cada uso de $L \to IL$ agrega una instrucción pendiente y la derivación solo termina al usar $L \to I$: no existe un camino que produzca una lista vacía. Por eso `Begin Programa prueba End` se rechaza, pero un programa con una única asignación se acepta.

Las dos asignaciones se distinguen por el contenido después de `=`. P5 produce exactamente un operando antes de `#`; P6 exige dos operandos separados por un operador. Cada V se reduce a un solo `ID` o `NUM`: no puede absorber el símbolo `+` ni una expresión completa. Después del primer operando, un `#` corresponde a la forma simple y uno de los cinco operadores corresponde a la forma aritmética. Por ejemplo:

Cadena	Producción central	Por qué corresponde
`temporal = x #`	$C \to D = V\#$	Después de `x` viene el terminador; hay un solo valor a la derecha de la asignación.
`m = k% 10 #`	$R \to D = VFV\#$	`k` y `10` ocupan los dos V; `%` ocupa F.
`salida m #`	$O \to$ `salida` $D\#$	Empieza con una reservada y no contiene asignación ni operación.

Las producciones restringen también las formas que no se deben aceptar. El destino de ambas asignaciones es D, por lo que `10 = x #` no se genera. La salida exige D y no V, de modo que `salida 20 #` queda fuera del lenguaje definido. Ninguna instrucción omite `#`. Además, P1 coloca una única lista entre el nombre y `End`; no permite una instrucción antes del encabezado ni después del cierre.

Toda estructura del alcance tiene una construcción y toda derivación respeta ese alcance. Para construir un programa se toma P1, se genera su nombre con P10, se elige la longitud de la lista con P2 y se deriva cada instrucción mediante P3 a P11. Recíprocamente, una derivación terminal desde S debe usar ese encabezado y ese cierre, una lista no vacía y únicamente las tres formas de instrucción descritas. Esta correspondencia explica por qué las reglas cubren el lenguaje definido; las pruebas comprueban después ejemplos concretos y errores de sus límites.

c) Verificación del programa completo de referencia

La gramática representa el programa completo. Su encabezado aplica P1: `operativo_01_02` es `ID`; el comentario se omite antes del análisis sintáctico. Para sus 20 instrucciones se usa $L \to IL$ diecinueve veces y luego $L \to I$. Cada I toma la alternativa indicada:

N.º	Instrucciones del programa	Rama
1–2	`x = 1.345 #; y = 20 #`	$I \Rightarrow A \Rightarrow C$
3–4	`z = x + y #; z2 = x + y #`	$I \Rightarrow A \Rightarrow R$
5–6	`h = z - x #; h2 = z - x #`	$I \Rightarrow A \Rightarrow R$
7–9	`k = h * y #; t = k / z #; m = k% 10 #`	$I \Rightarrow A \Rightarrow R$
10–14	`salida z #; salida h #; salida k #; salida t #; salida m #`	$I \Rightarrow O$
15–18	`temporal = 0 #; temporal = x #; x = y #; y = temporal #`	$I \Rightarrow A \Rightarrow C$
19–20	`salida x #; salida y #`	$I \Rightarrow O$

La expansión de la lista es

$$L \Rightarrow IL \Rightarrow IIL \Rightarrow \cdots \Rightarrow I^{19}L \Rightarrow I^{20}.$$

Esta expresión describe solo la lista. En una derivación por la izquierda del programa entero se completa cada I antes de expandir el L que queda a su derecha. Para la primera instrucción, el prefijo de esa derivación es:

$$\begin{aligned} S &\Rightarrow \texttt{Begin Programa } D\ L \texttt{ End} \\ &\Rightarrow \texttt{Begin Programa ID[operativo_01_02] } L \texttt{ End} \\ &\Rightarrow \texttt{Begin Programa ID[operativo_01_02] } I\ L \texttt{ End} \\ &\Rightarrow \texttt{Begin Programa ID[operativo_01_02] } A\ L \texttt{ End} \\ &\Rightarrow \texttt{Begin Programa ID[operativo_01_02] } C\ L \texttt{ End} \\ &\Rightarrow \texttt{Begin Programa ID[operativo_01_02] } D = V\#\ L \texttt{ End}. \end{aligned}$$

Luego $D \Rightarrow \text{ID}[\texttt{x}]$ y $V \Rightarrow N \Rightarrow \text{NUM}[\texttt{1.345}]$ completan la primera asignación. Se continúa con las otras diecinueve instrucciones y se consume `End`.

La comprobación construyó un árbol con **20 instrucciones, 13 asignaciones y 7 salidas**. Seis asignaciones usan P5 y siete usan P6. Dos reconocedores en Python coinciden y el CYK de JFLAP acepta también el programa tokenizado. La fuente íntegra está en `ejemplos/programa_referencia.txt`; su resultado se registra como caso `REF` en las evidencias.

La derivación completa del programa se adjunta en el repositorio: incluye 142 aplicaciones de producciones desde S hasta la última forma terminal, sin omitir instrucciones intermedias. El archivo JSON del mismo nombre conserva el árbol y los lexemas con sus posiciones. El verificador comprueba que cada sustitución use una regla de P, que se expanda el no terminal más a la izquierda y que la forma final coincida con todos los tokens de la fuente.

2.3. Pregunta 3 Derivaciones y árbol sintáctico

a) Derivación por la izquierda de `z = x + y #`

Partimos de I, porque la cadena es una instrucción. Cada fila reemplaza únicamente el no terminal ubicado más a la izquierda. Se usan los códigos de JFLAP para verificar cada sustitución: `i` es `ID` y conserva los lexemas `z`, `x` e `y` en ese orden.

Paso	Producción aplicada	Forma sentencial
0	Inicio	`I`
1	I → `A`	`A`
2	A → `R`	`R`
3	R → `D=VFV#`	`D=VFV#`
4	D → `i`	`i=VFV#`
5	V → `D`	`i=DFV#`
6	D → `i`	`i=iFV#`
7	F → `+`	`i=i+V#`
8	V → `D`	`i=i+D#`
9	D → `i`	`i=i+i#`

Con los atributos de los identificadores, la misma derivación se lee:

$$\begin{aligned} I &\Rightarrow A \Rightarrow R \Rightarrow D = VFV\# \\ &\Rightarrow \mathrm{ID}[\mathtt{z}] = VFV\# \Rightarrow \mathrm{ID}[\mathtt{z}] = DFV\# \\ &\Rightarrow \mathrm{ID}[\mathtt{z}] = \mathrm{ID}[\mathtt{x}]FV\# \Rightarrow \mathrm{ID}[\mathtt{z}] = \mathrm{ID}[\mathtt{x}] + V\# \\ &\Rightarrow \mathrm{ID}[\mathtt{z}] = \mathrm{ID}[\mathtt{x}] + D\# \Rightarrow \mathrm{ID}[\mathtt{z}] = \mathrm{ID}[\mathtt{x}] + \mathrm{ID}[\mathtt{y}]\#. \end{aligned}$$

La secuencia terminal es `ID = ID + ID #`; sus lexemas reconstruyen exactamente `z = x + y #`. No se introduce una regla `ID`→`z`: `ID` ya es terminal y `z` es su atributo léxico. Así se verifica la cadena solicitada sin enumerar los posibles identificadores dentro de P.

b) Derivación por la izquierda de `salida z #`

Paso	Producción aplicada	Forma sentencial
0	Inicio	`I`
1	I → `O`	`O`
2	O → `sD#`	`sD#`
3	D → `i`	`si#`

Con lexemas: $I \Rightarrow O \Rightarrow$ `salida` $D\ \# \Rightarrow$ `salida` ID[`z`] #. Se obtiene `salida z #`; en JFLAP se escribe `si#`. El terminador forma parte de P9 y no se puede omitir.

c) Árbol de derivación de `z = x + y #`

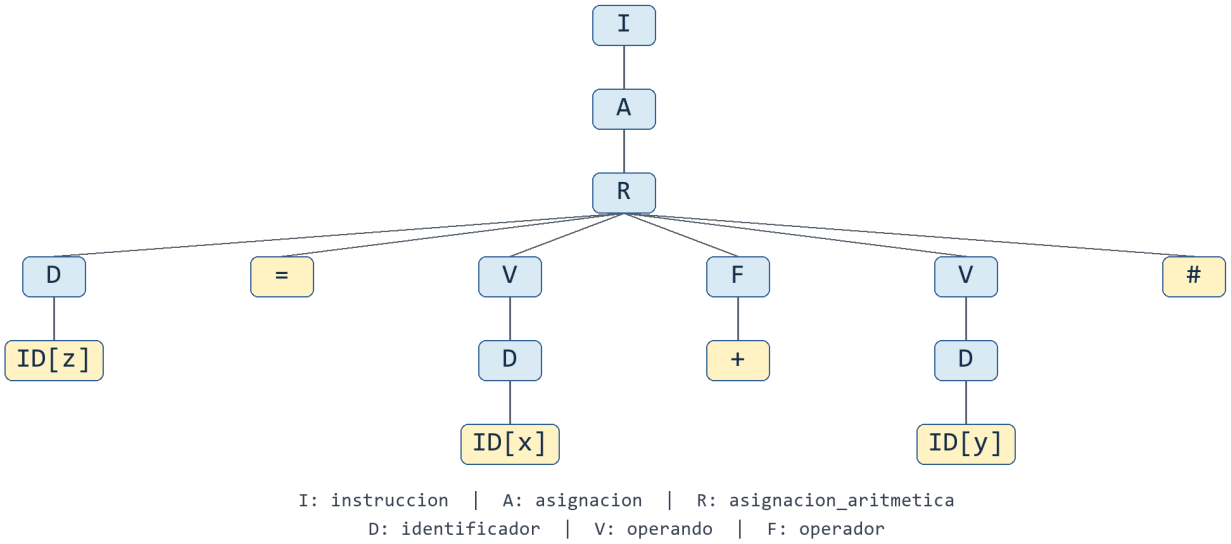


Figura 1: Árbol base con raíz I. Las hojas `ID` conservan los lexemas `z`, `x` e `y`; su lectura de izquierda a derecha reconstruye la instrucción.

d) Relación entre raíz, no terminales, terminales y hojas

La raíz I indica que se analiza una instrucción. Sus nodos internos A y R seleccionan la asignación aritmética. Los seis hijos de R son D, `=`, V, F, V y `#`, exactamente como en P6. El primer D representa el destino `z`; los dos V generan `x` e `y`; y F genera `+`.

Los no terminales son categorías que deben expandirse. Las hojas son terminales: ID[`z`], `=`, ID[`x`], `+`, ID[`y`] y `#`. Al recorrerlas de izquierda a derecha se obtiene el rendimiento del árbol. En un programa completo la raíz sería S; aquí se usa el subárbol desde I. La copia de JFLAP para instrucciones agrega la raíz auxiliar $S \to I$, documentada en el anexo A.

e) Validación de las tres cadenas adicionales

Cadena	**¿Desde I?**	**Justificación**
`x = 20 #`	Sí	$I \Rightarrow A \Rightarrow C \Rightarrow D = V\#$. Se completa con $D \Rightarrow$ ID[`x`] y $V \Rightarrow N \Rightarrow$ NUM[`20`].
`z = x * y #`	Sí	$I \Rightarrow A \Rightarrow R \Rightarrow D = VFV\#$. Los identificadores conservan `z`, `x`, `y`, y $F \Rightarrow$ `*`.
`salida temporal #`	Sí	$I \Rightarrow O \Rightarrow$ `salida` D #, con $D \Rightarrow$ ID[`temporal`]. El nombre cumple el patrón de identificadores.

Estas justificaciones resumen derivaciones de pertenencia. Los tres casos también fueron verificados por los dos reconocedores de Python y por JFLAP.

2.4. Pregunta 4 Validación y detección de errores

La pertenencia se evalúa desde I en los casos 1 a 7 y desde S en el caso 8. Así conservamos el inicio de programa y analizamos las instrucciones aisladas de la actividad.

Caso	Cadena	¿Se genera?	Justificación o error detectado
1	`x = 10 #`	Sí, desde I	Asignación simple: `ID = NUM #`. P5 admite el número mediante $V \to N \to$ `NUM`.
2	`z = x + y #`	Sí, desde I	Tiene destino, asignación, dos operandos, operador infijo y terminador. Sigue P6, con $F \to$ `+`.
3	`salida z #`	Sí, desde I	Sigue P9: palabra reservada, identificador y terminador.
4	`x 10 #`	No	Ambas asignaciones exigen `=` después del destino. El número aparece antes de ese símbolo. Corrección mínima: `x = 10 #`.
5	`z = + x y #`	No	Después de `=` debe comenzar V, que solo genera `ID` o `NUM`; `+` no es operando. El operador debe ser infijo. Corrección: `z = x + y #`.
6	`salida #`	No	P9 exige D antes de `#`; falta la variable que se quiere mostrar. Corrección posible: `salida z #`.
7	`x = 20`	No	Falta `#`. No se acepta la entrada al agotarse antes del terminador. Corrección: `x = 20 #`.

8	`Begin`
	`Programa prueba`
	`x = 5 #`
	`salida x #`
	`End`
		Sí, desde S	P1 usa el nombre `prueba`, $L \Rightarrow IL \Rightarrow II$, una asignación simple y una salida. El cierre `End` aparece después de ambas instrucciones.

Interpretación de los separadores del caso 8

La pauta imprime `Begin / Programa prueba / x = 5 # / salida x # / End`. Interpretamos esas barras como separadores de líneas, coherentes con el programa de referencia y con las barras que separan ejemplos en la extensión. Arriba se muestra en cinco líneas. Si se leen literalmente como operadores de división, la cadena es **inválida**: P1 exige `Programa` después de `Begin`, sin `/`. Comprobamos ambas lecturas y registramos el rechazo de la versión literal como `P4_8_BARRAS_LITERALES`.

Análisis general

Los cuatro errores obligatorios son sintácticos: sus lexemas individuales son reconocibles, pero faltan símbolos o están en un orden no permitido. En el caso 4 falta la asignación; en el 5 se usa una disposición prefija que no definimos; en el 6 falta un identificador; y en el 7 falta el cierre de instrucción. Las correcciones reconstruyen cadenas que siguen las reglas existentes, sin modificar las producciones para acomodar errores.

3. Extensión de la gramática

Construcción seleccionada y nueva sintaxis

Elegimos la opción C: **declaración de variables**, para separar la indicación de un tipo de la asignación de un valor. Definimos dos formas, donde `ID` representa un identificador:

```
entero ID #    real ID #
```

Por ejemplo, `entero contador #` declara una variable de tipo entero y `real temperatura #` una de tipo real. La GLC comprueba la forma de la declaración; no mantiene una tabla de símbolos ni exige que toda variable esté declarada.

Modificaciones formales

Elemento	Modificación
Nuevos terminales	`entero` y `real`, reservadas en la versión extendida.
Nuevos no terminales	$J =$ `<declaracion>` y $T =$ `<tipo>`.
Alternativa añadida	$I \to J$. Se conservan $I \to A$ e $I \to O$.
Nuevas producciones	$J \to T\ D$ `#`; $T \to$ `entero` | `real`.
Definición extendida	$G' = (V', \Sigma', P', S)$, con $V' = V \cup \{J, T\}$, $\Sigma' = \Sigma \cup \{\texttt{entero}, \texttt{real}\}$ y $P' = P \cup \{I \to J, J \to TD\#, T \to \texttt{entero}, T \to \texttt{real}\}$.

Hay trece no terminales, quince terminales y 23 alternativas. Como $L \to IL \mid I$ ya admite cualquier instrucción, no hay que cambiar la lista para mezclar declaraciones, asignaciones y salidas.

Cadena válida y cadena inválida

Válida: `entero contador #`. Aparecen un tipo reservado, un identificador y el terminador.

Inválida: `entero #`. Falta D después del tipo; `#` no puede ocupar la posición del nombre. Insertar `contador` corrige el error. También se rechaza `entero contador = 0 #`, porque no se definió inicialización dentro de una declaración.

Derivación por la izquierda del ejemplo válido

En JFLAP, `t` representa `entero` y `r` representa `real`:

Paso	Producción aplicada	Forma sentencial
0	Inicio	`I`
1	I → `J`	`J`
2	J → `TD#`	`TD#`
3	T → `t`	`tD#`
4	D → `i`	`ti#`

Con lexemas: $I \Rightarrow J \Rightarrow TD\# \Rightarrow$ `entero` $D\# \Rightarrow$ `entero` ID[`contador`] #. Para declarar un real cambia únicamente $T \to$ `real`.

Verificación en un programa completo

```
Begin
Programa ejemplo_tipos
entero contador #
real temperatura #
contador = 0 #
temperatura = 1.345 #
contador = contador + 1 #
salida contador #
salida temperatura #
End
```

El encabezado y el cierre siguen P1. Las primeras dos instrucciones usan $I \to J$; las asignaciones y salidas conservan las producciones anteriores. El caso `EXT_PROG` acepta este programa en Python y JFLAP. También se probaron `real temperatura #`, la declaración sin terminador, la inicialización no definida y el uso de `real` como nombre después de `entero`.

La inclusión $P \subset P'$ conserva las derivaciones sobre tokens de la base. En el código fuente hay un cambio léxico explícito: `entero` y `real`, que en la base podían clasificarse como `ID`, quedan reservadas en la extensión. Por ejemplo, `real = 1 #` deja de ser una asignación válida en la versión extendida. Esta reserva es una decisión de la nueva sintaxis.

4. Conclusiones

La construcción mostró que reconocer lexemas válidos no basta para aceptar una instrucción: también hay que exigir su orden y sus delimitadores. Los casos `x 10 #`, `z = + x y #`, `salida #` y `x = 20` contienen elementos conocidos, pero cada uno incumple una producción concreta. Separar asignaciones simples, asignaciones aritméticas y salidas permitió explicar esos errores y relacionarlos con el árbol. También fue necesario distinguir una instrucción aislada de un programa completo, porque no comparten el mismo punto de inicio.

La gramática representa las 20 instrucciones del programa de referencia y admite nombres y números nuevos mediante categorías léxicas. Las derivaciones por la izquierda y el recorrido de las hojas llegan a los mismos tokens, y las verificaciones de Python y JFLAP coinciden en los casos probados. La extensión agrega dos tipos mediante cuatro alternativas nuevas y conserva las formas anteriores de instrucción. Su aceptación acredita corrección sintáctica; no demuestra que las variables estén inicializadas, que sus tipos sean compatibles o que una división sea ejecutable. Esas comprobaciones requieren una etapa semántica que no forma parte del laboratorio.

5. Consideraciones de entrega

Se entrega este informe en PDF, manteniendo la numeración de la plantilla, junto con los archivos de JFLAP y las evidencias en el repositorio público:

github.com/Frante23/info1148-lab3

Archivo final: `Lab_03_JefeGrupo_vicente_rivera.pdf`. El repositorio contiene las gramáticas base y extendida, sus copias para instrucciones, la fuente del informe, ejemplos y resultados verificables. Los anexos documentan la codificación y las vistas de JFLAP; complementan las explicaciones de las preguntas.

A. Codificación y reproducción de las evidencias

Equivalencias entre el informe y JFLAP

Las mayúsculas representan los once no terminales base: `S,L,I,A,C,R,V,F,O,D,N`. La extensión agrega `J,T`. Las minúsculas siguientes codifican tokens, no lexemas de variables del programa:

Código	Terminal	Código	Terminal
`b`	`Begin`	`p`	`Programa`
`e`	`End`	`s`	`salida`
`i`	`ID`	`n`	`NUM`
`t`	`entero` (extensión)	`r`	`real` (extensión)
`=`, `#`, `+`, `-`, `*`, `/` y `%` se conservan sin cambio.			

La correspondencia es uno a uno sobre tipos de token. No distingue `x` de `y`: su lexema se conserva como atributo. La GLC comprueba su categoría y ubicación.

Archivos que se deben abrir

Archivo en `gramaticas/`	Uso
`gramatica_base.jff`	Programas completos, raíz S y P1 original.
`gramatica_extendida.jff`	Programas completos con declaraciones.
`gramatica_base_instrucciones.jff`	Instrucciones aisladas de las preguntas 3 y 4.
`gramatica_extendida_instrucciones.jff`	Instrucciones aisladas con declaraciones.

Adaptador de inicio: JFLAP 7.1 conserva S al abrir estos XML. En las copias para instrucciones se reemplaza solo $S \to bpDLe$ por $S \to I$. El subárbol desde I conserva las producciones del informe. No se agrega $S \to I$ a la gramática principal de programas.

Para reproducir la asignación: abrir `gramatica_base_instrucciones.jff`, elegir **Input → Brute Force Parse**, ingresar `i=i+i#`, iniciar el análisis y avanzar hasta completar el árbol. Para la salida usar `si#`; para la declaración abrir la copia extendida e ingresar `ti#`. El TSV `entradas_jflap.tsv` conserva la entrada codificada de cada caso.

Comprobaciones realizadas

El corpus incluye 82 casos: 43 aceptados y 39 rechazados. Diez rechazos son léxicos; las 72 entradas restantes se contrastaron con JFLAP. La vacía se rechaza porque no hay reglas que generen ε. Se comprueban los cuatro XML, cada producción aplicada, el orden de sustitución por la izquierda y la coincidencia entre hojas y tokens. También se obtuvieron tres árboles con el `BruteParser` original.

El script Java elimina producciones unitarias y convierte a forma normal de Chomsky *en memoria*, con los convertidores de JFLAP, antes de ejecutar su CYK. Los `.jff` conservan las reglas originales; sus árboles no se sustituyen por los de la forma normal. Las vistas se renderizaron desde componentes originales de JFLAP con los archivos reabiertos. Las pruebas verifican los casos ensayados; la explicación de las producciones justifica el comportamiento general.

B. Vistas de las gramáticas en JFLAP

JFLAP 7.1 | gramatica_base.jff

S	⟶	bpDLe
L	⟶	IL
L	⟶	I
I	⟶	A
I	⟶	O
A	⟶	C
A	⟶	R
C	⟶	D=V#
R	⟶	D=VFV#
V	⟶	D
V	⟶	N
F	⟶	+
F	⟶	-
F	⟶	*
F	⟶	/
F	⟶	%
O	⟶	sD#
D	⟶	i
N	⟶	n

Figura 2: Tabla original de JFLAP 7.1 al abrir `gramatica_base.jff`, con las 19 alternativas del informe.

JFLAP 7.1 | gramatica_extendida.jff

S	⟶	bpDLe
L	⟶	IL
L	⟶	I
I	⟶	A
I	⟶	O
I	⟶	J
A	⟶	C
A	⟶	R
C	⟶	D=V#
R	⟶	D=VFV#
V	⟶	D
V	⟶	N
F	⟶	+
F	⟶	-
F	⟶	*
F	⟶	/
F	⟶	%
O	⟶	sD#
D	⟶	i
N	⟶	n
J	⟶	TD#
T	⟶	t
T	⟶	r

Figura 3: Tabla original de `gramatica_extendida.jff`: se añaden $I \to J$, $J \to TD\#$ y dos alternativas de T.

C. Árboles obtenidos con el analizador original de JFLAP

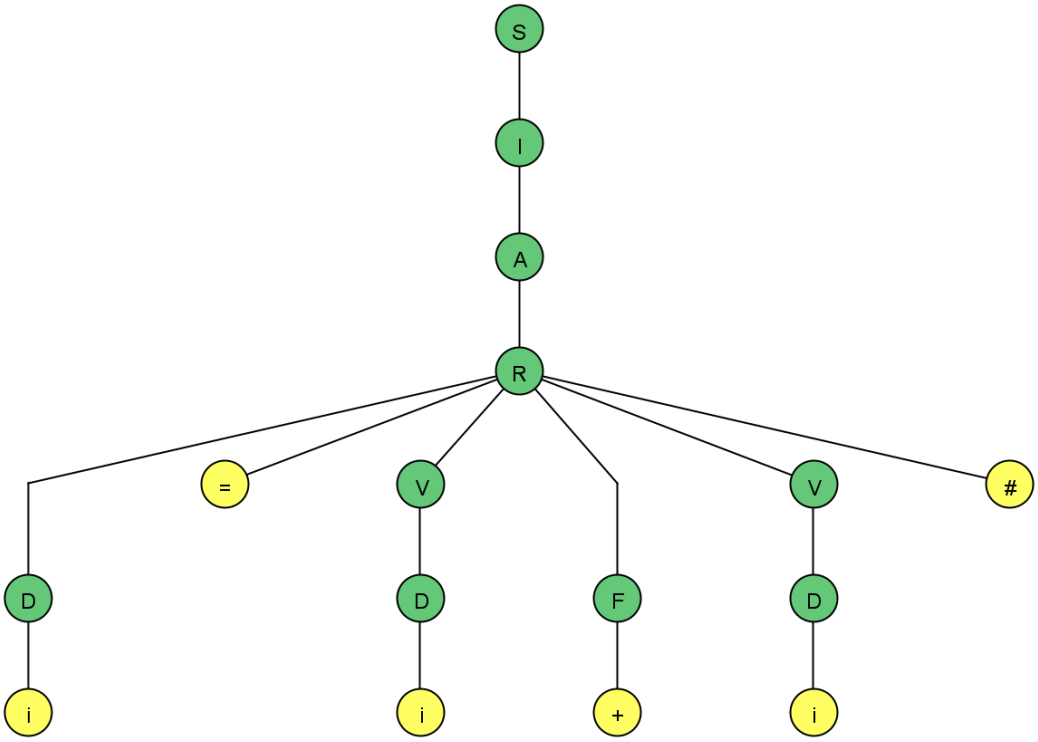


Figura 4: Árbol de `i=i+i#`, correspondiente a `z = x + y #`. $S \rightarrow I$ es el adaptador de la copia para instrucciones. Los nodos amarillos son terminales; los verdes, no terminales.

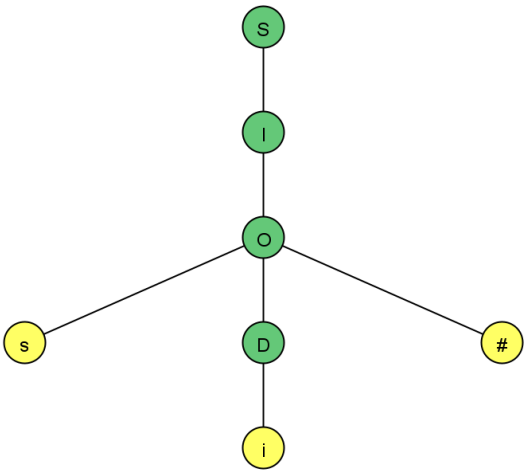


Figura 5: Árbol de `si#`: salida de un identificador, con el lexema `z` en la cadena solicitada.

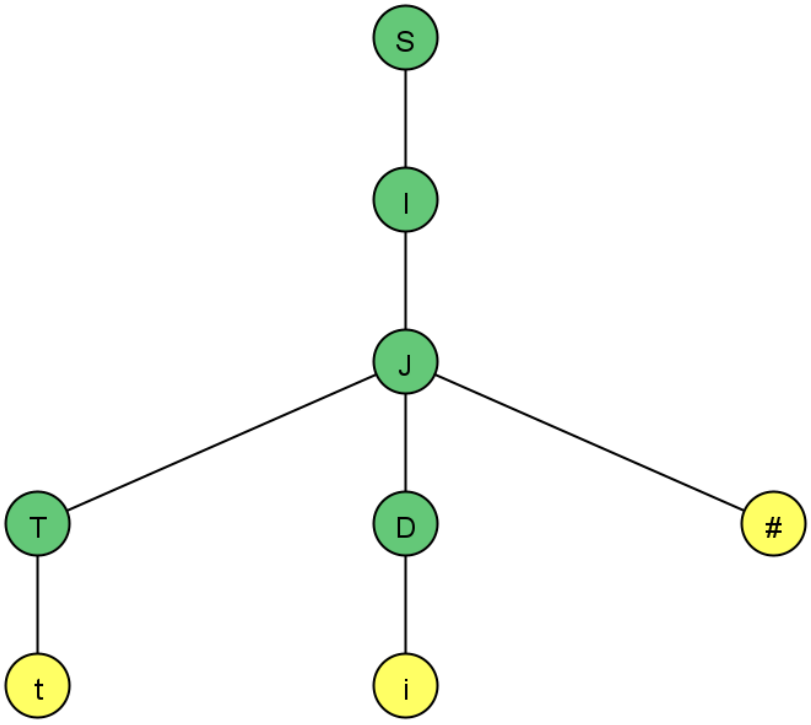


Figura 6: Árbol de `ti#` con la gramática extendida: declaración `entero contador #`.

Fuentes de la actividad

El desarrollo sigue el enunciado *Laboratorio 3 Diseño y Validación de Gramáticas*, la *Rúbrica de evaluación del Laboratorio 3* y la plantilla *Formato Informe LAB 3*, proporcionados para INFO1148. Se conserva la división de las cuatro preguntas, la extensión creativa y la numeración solicitada. Las dos copias recibidas de la rúbrica tienen el mismo contenido.